Programmazione parallela su architetture GPU

Appunti personali

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Triennale in Informatica (L-31)

Autori
Emanuele Galiano

Anno accademico
Anno Accademico 2025/2026

Versione: 5 marzo 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Programmazione parallela su architetture GPU– Appunti personali* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania tenuto dal Prof. Giuseppe Bilotta nell'anno accademico Anno Accademico 2025/2026. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in unico documento.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/GPU-Parallel-Programming`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Programmazione parallela su architetture GPU – Appunti personali*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2025/2026

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole

e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**
Diritti concessi v
Condizioni v
Assenza di garanzia vi
Testo completo della licenza vi

1 Introduzione **1**
1.1 Calcolo parallelo 1
1.2 Limiti del calcolo parallelo 1
1.2.1 Caso ideale di parallelizzazione 1
1.2.2 Legge di Amdahl 2
1.2.3 Legge di Gustafson 3
1.3 Tipi di parallelismo 4
1.3.1 Task-based parallelism 4
1.3.2 Data-based parallelism 5
1.4 Hardware per il calcolo parallelo 6

Capitolo 1

Introduzione

1.1 Calcolo parallelo

Il calcolo parallelo è un paradigma di programmazione che consente di eseguire più operazioni contemporaneamente, sfruttando la potenza di elaborazione di più unità di calcolo. A differenza dello pseudoparallelismo, che simula l'esecuzione simultanea di più processi su un singolo processore, il calcolo parallelo utilizza effettivamente più processori o core per eseguire compiti in parallelo.

Questo paradigma è utile perché riduce i tempi di esecuzione e aumenta la dimensione dei dati processabili, consentendo di affrontare problemi più complessi e di ottenere risultati più rapidamente.

1.2 Limiti del calcolo parallelo

1.2.1 Caso ideale di parallelizzazione

Per comprendere i limiti del calcolo parallelo è utile partire da un modello ideale. Supponiamo che un programma sia completamente parallelizzabile, cioè che tutto il lavoro possa essere suddiviso tra più unità di calcolo senza alcuna parte sequenziale.

Se T_0 indica il tempo di esecuzione del programma in modalità sequenziale e N il numero di unità di calcolo disponibili, il tempo di esecuzione ideale del programma parallelo sarebbe:

$$T_L(N) = \frac{T_0}{N}$$

In questo caso il tempo di esecuzione diminuisce linearmente all'aumentare del numero di processori. Lo speed-up ottenuto è quindi:

$$S_L(N) = \frac{T_0}{T_L(N)} = N$$

Questo significa che, in condizioni ideali, raddoppiare il numero di unità di calcolo dimezza il tempo di esecuzione. In tale modello non esiste quindi un limite teorico allo speed-up ottenibile aumentando il numero di processori.

1.2.2 Legge di Amdahl

Il calcolo parallelo non può essere applicato a tutti i problemi, e anche quando è possibile, esiste un limite alla velocità di esecuzione che può essere raggiunta. Questo limite è descritto dalla legge di Amdahl:

> *La velocità massima di un programma parallelo è limitata dalla frazione di codice che deve essere eseguita in modo sequenziale.*

Adamahl ha dimostrato che se una frazione p del programma può essere parallelizzata, allora la velocità massima teorica che si può ottenere è data da:

$$T_A(N) = \frac{pT_0}{N} + (1-p)T_0$$

dove T_0 è il tempo di esecuzione del programma sequenziale e N è il numero di processori. Si può anche esprimere il massimo speed-up come:

$$S_A(N) = \frac{T_0}{T_A(N)} = \frac{1}{p/N + (1-p)} = \frac{N}{p + N(1-p)}$$

Da questa formula si evince che, anche con un numero infinito di processori, il massimo speed-up è

$$\lim_{N\to\infty} S_A(N) = \frac{1}{1-p}$$

Il che significa che se una parte significativa del programma è sequenziale, il miglioramento ottenuto con il calcolo parallelo sarà limitato, indipendentemente dal numero di processori utilizzati.

p	2	4	100	200	400	∞
10%	1.05	1.08	1.11	1.11	1.11	1.11
25%	1.14	1.23	1.33	1.33	1.33	1.33
50%	1.33	1.60	1.98	1.99	1.99	2.00
75%	1.60	2.29	3.88	3.94	3.97	4.00
99%	1.98	3.88	50.2	66.9	80.2	100

Tabella 1.1: Speed-up teorico secondo la legge di Amdahl al variare della frazione parallelizzabile p e del numero di processori. Come si può notare all'aumentare delle unità di calcolo, lo speed-up tende a un limite massimo che dipende dalla frazione di codice sequenziale.

Ottimizzazione della legge di Amdahl. Ipotizzando che il lavoro totale possa essere diviso in parti multiple $p_1, p_2, \ldots, p_r$ e ognuna abbia uno speed-up $s_1, s_2, \ldots, s_r$, allora lo speed-up totale è dato da:

$$S = \frac{1}{\sum_{i=1}^{r} \frac{p_i}{s_i}}$$

In questo caso ci si potrebbe chiedere se convenga concentrarsi sull'ottimizzazione della parte con lo speed-up maggiore s_i oppure su quella che rappresenta la porzione più grande del programma p_i.

Consideriamo ad esempio un programma suddiviso in due parti, con $p_1 = 0.25$ e $p_2 = 0.75$. Supponiamo inoltre che la prima parte possa essere accelerata con uno speed-up $s_1 = 20$, mentre la seconda con uno speed-up $s_2 = 5$.

Se si ottimizza solo la prima parte si ottiene:

$$S = \frac{1}{\frac{0.25}{20} + 0.75} \approx 1.31$$

Se invece si ottimizza solo la seconda parte:

$$S = \frac{1}{0.25 + \frac{0.75}{5}} = 2.5$$

Infine, ottimizzando entrambe le parti:

$$S = \frac{1}{\frac{0.25}{20} + \frac{0.75}{5}} \approx 6.15$$

Questo esempio mostra che, per ottenere il miglior miglioramento complessivo, conviene concentrarsi sull'ottimizzazione della parte che occupa la porzione maggiore del tempo di esecuzione del programma.

1.2.3 Legge di Gustafson

La legge di Amdahl si concetra su quanto è possibile massimizzare lo speed-up assumendo che il lavoro sia fisso (strong scaling).

La legge di Gustafson, invece, si concentra su quanto è possibile aumentare la dimensione del lavoro mantenendo costante il tempo di esecuzione (weak scaling).

> *Se una frazione p del lavoro può avere uno speed-up di fattore s, allora il lavoro totale che può essere eseguito in un tempo costante è dato da:*
>
> $$S_G = \frac{W_S}{W_0} = 1 - p + sp$$
>
> dove W_S è il lavoro totale eseguito in parallelo, W_0 è il lavoro eseguito in modalità sequenziale.

Esempio. Supponiamo che l'80% del lavoro possa essere parallelizzato con uno speed-up di 20, allora nello stesso tempo di esecuzione del programma sequenziale, è possibile eseguire un lavoro totale di:

$$S_G = 1 - 0.8 + 20 \cdot 0.8 = 16.2$$

Non linearità dello speed-up. La legge di Gustafson mostra che, aumentando la dimensione del lavoro, è possibile ottenere uno speed-up maggiore, anche se la frazione di codice parallelizzabile rimane costante. Tuttavia, è importante notare che lo speed-up non aumenta linearmente con la dimensione del lavoro, ma dipende dalla frazione di codice parallelizzabile e dallo speed-up ottenuto su quella frazione.

Esempio. Supponiamo che W sia $O(D^3)$ (dove D è la dimensione dei dati). Allora $16\times$ più lavoro corrisponde a un aumento della dimensione dei dati di $\sqrt[3]{16} \approx 2.52$ volte.

1.3 Tipi di parallelismo

Un problema può essere parallelizzato in diversi modi, a seconda della natura del lavoro da svolgere e delle dipendenze tra le operazioni. I principali tipi di parallelismo sono:

- Data-based: le operazioni possono essere eseguite in parallelo su diversi dati, ad esempio l'elaborazione di un array o di una matrice.
- Task-based: le operazioni possono essere eseguite in parallelo su diversi compiti, ad esempio l'esecuzione di più funzioni o processi indipendenti.

1.3.1 Task-based parallelism

Questo tipo di parallelismo si basa sull'esecuzione di compiti indipendenti in parallelo. Ad esempio, se un programma deve eseguire più funzioni che non dipendono l'una dall'altra, queste possono essere eseguite contemporaneamente su processori diversi. Si preferisce questo tipo di parallelismo se:

- Il problema può essere suddiviso in compiti (meglio indipendenti e di dimensione simile).
- Numero di task $\approx$ numero di processori disponibili.
- Le unità che possiedono diversi task possono comunicare in modo semplice.

Esempio: pipeline di elaborazione. Consideriamo un carico di lavoro composto da tre tipi di task applicati a più dataset (A, B, C, ...): la **lettura** dei dati (R), l'**elaborazione** o calcolo (C) e la **scrittura** dei risultati (W). Per semplicità assumiamo tempi di esecuzione fittizi costanti pari a $t_R = 2$ per la lettura, $t_C = 3$ per l'elaborazione e $t_W = 2$ per la scrittura. Nel caso **dipendente** ogni dataset deve rispettare la sequenza $R \to C \to W$, cioè i dati devono essere letti prima di poter essere elaborati e i risultati possono essere scritti solo dopo il completamento del calcolo. Nel caso **indipendente**, invece, i task possono essere schedulati senza vincoli di dipendenza e quindi distribuiti più liberamente tra le unità di calcolo.

Nel caso più semplice, mostrato in Figura 1.1, l'esecuzione avviene su una sola unità di calcolo. Tutti i task vengono quindi eseguiti in modo completamente seriale e ogni dataset attraversa l'intera sequenza di operazioni prima che il successivo possa iniziare. L'ordine di esecuzione diventa quindi $R_A, C_A, W_A, R_B, C_B, W_B, \ldots$, rappresentando il caso base senza alcuna forma di parallelismo.

Introducendo una seconda unità di calcolo e mantenendo le dipendenze tra task (Figura 1.2), è possibile ottenere una parziale sovrapposizione delle operazioni. Ad esempio, mentre una unità legge il dataset successivo, l'altra può elaborare quello precedente. Tuttavia la presenza delle dipendenze impone comunque dei vincoli sull'ordine di esecuzione e genera inevitabilmente alcuni periodi di inattività.

Se i task sono indipendenti (Figura 1.3), lo scheduler può distribuire le operazioni tra le due unità con maggiore libertà. Questo permette di riempire meglio i tempi morti e di ottenere

un utilizzo più efficiente delle risorse di calcolo, aumentando quindi il grado di parallelismo effettivamente sfruttabile.

Aumentando il numero di dataset, come mostrato in Figura 1.4, la pipeline tende a stabilizzarsi anche nel caso dipendente. Le unità restano occupate più a lungo e il throughput complessivo migliora, ma il cammino critico imposto dalle dipendenze continua comunque a limitare lo speed-up massimo ottenibile.

Utilizzando tre unità di calcolo con task dipendenti (Figura 1.5) l'aggiunta di risorse non produce necessariamente un'accelerazione proporzionale. Il vincolo $R \rightarrow C \rightarrow W$ per ogni dataset rende infatti difficile mantenere tutte le unità occupate contemporaneamente, causando possibili squilibri nel carico di lavoro.

Nel caso indipendente con tre unità (Figura 1.6), invece, il parallelismo può essere sfruttato in modo più efficace. I task possono essere distribuiti dinamicamente tra le unità e il carico di lavoro risulta più bilanciato, permettendo di avvicinarsi ad un utilizzo più efficiente delle risorse disponibili.

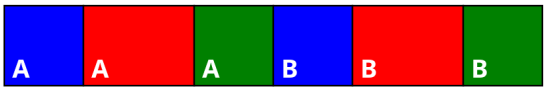

Figura 1.1: Task-based: una sola unità (esecuzione seriale).

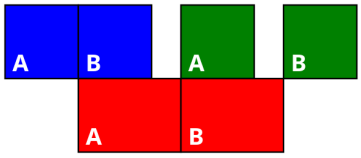

Figura 1.2: Task-based: due unità, caso dipendente.

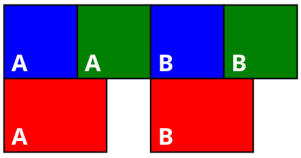

Figura 1.3: Task-based: due unità, caso indipendente.

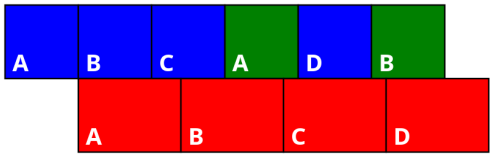

Figura 1.4: Task-based: due unità, più lavoro, caso dipendente.

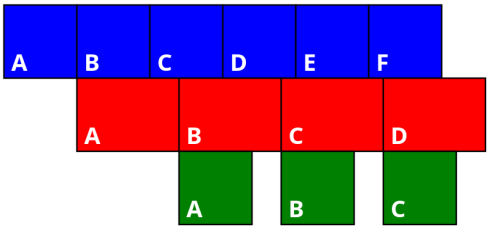

Figura 1.5: Task-based: tre unità, più lavoro, caso dipendente.

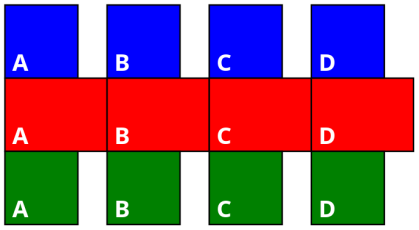

Figura 1.6: Task-based: tre unità, più lavoro, caso indipendente.

1.3.2 Data-based parallelism

Il data-based parallelism si basa sull'esecuzione di operazioni in parallelo su diversi dati. Tutte le unità computazionali fanno lo stesso lavoro, la divisione del lavoro avviene sui dati. Si preferisce questo approccio quando:

- Il lavoro può essere suddiviso in operazioni simili e semplici da eseguire su diversi dati.
- Si deve lavorare con una grossa mole di dati che può essere facilmente suddivisa in blocchi indipendenti.

Esempio: somma vettoriale. Un esempio classico di parallelismo *data-based* è la somma vettoriale. Consideriamo due vettori A e B di lunghezza N e un vettore risultato C, tale che:

$$C_i = A_i + B_i \qquad i = 0, \ldots, N-1$$

In questo caso l'operazione da eseguire è la stessa per tutti gli elementi, mentre cambiano solo i dati su cui essa viene applicata. Questo rende il problema particolarmente adatto al parallelismo data-based, poiché ogni elemento del vettore può essere elaborato indipendentemente dagli altri.

Seriale. - Nel caso **seriale** (Figura 1.7) un'unica unità di calcolo esegue tutte le operazioni in sequenza. Per ogni indice i vengono letti gli elementi A_i e B_i, viene calcolata la somma e il risultato viene scritto nella posizione corrispondente del vettore C. L'intero vettore viene quindi processato elemento per elemento.

Parallelo. - Nel caso **parallelo** (Figura 1.8), invece, più unità di calcolo eseguono la stessa operazione contemporaneamente su elementi diversi dei vettori. Ogni unità si occupa di un indice differente i, leggendo A_i e B_i, calcolando la somma e scrivendo il risultato in C_i. Poiché non esistono dipendenze tra le operazioni sui diversi elementi, tutte le somme possono essere eseguite in parallelo.

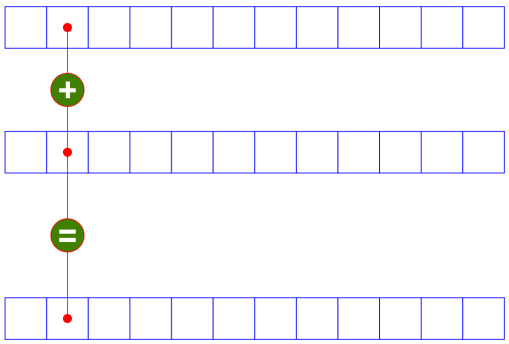

Figura 1.7: Somma vettoriale eseguita in modo seriale.

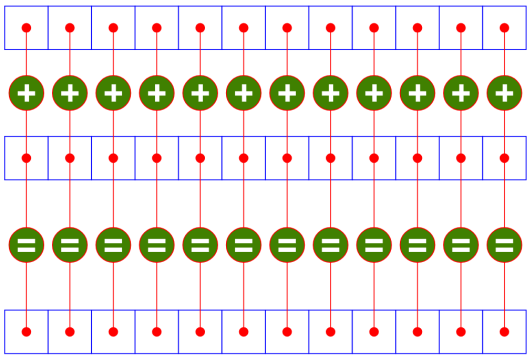

Figura 1.8: Somma vettoriale con parallelismo data-based.

1.4 Hardware per il calcolo parallelo